

Esercizio B1

Il problema è una leggera variante del problema di Hateville, in cui bisogna contare invece che cercare il massimo.

Sia $DP[n]$ il numero di stringhe di n bit che non contengono due 1 consecutivi; può essere calcolato ricorsivamente nel modo seguente:

$$DP[n] = \begin{cases} 1 & n = 0 \\ 2 & n = 1 \\ DP[n-1] + DP[n-2] & n \geq 2 \end{cases}$$

Nel caso generale, se piazzo un 1 come ultimo bit, poi dovrò piazzare un 0; conto quindi le combinazioni con $n-2$ bit. Se piazzo uno 0 come ultimo bit, non ho vincoli e quindi conto le combinazioni con $n-1$ bit. Questi due valori vanno sommati insieme, essendo esclusivi.

```
int bits(int n)
{
    int[] DP = new int[0...n]
    DP[0] = 1
    DP[1] = 2
    for i = 2 to n do
        DP[i] = DP[i-1] + DP[i-2]
    return DP[n]
}
```

Ancora una volta, quello che viene generata è la successione di Fibonacci.

L'algoritmo richiede $O(n)$ somme. Si noti tuttavia che n non è la dimensione del problema, è l'input del problema. Secondo il modello di costo logaritmico, il vero costo è $O(n^2)$, in quanto il numero di bit per rappresentare $\text{bits}(n)$ (e quindi il numero di operazioni per effettuare le somme) cresce linearmente con n .

Esercizio B2

Il problema si risolve tramite backtrack. Oltre alle stringhe T e P , viene passato uno stack.

Vengono passati due indici i e j , che vengono utilizzati per indicare il prefisso i -esimo di T e il prefisso j -esimo di T . Se $j = 0$, allora non ci sono più caratteri nel pattern, e quindi bisogna stampare lo stack (in ordine LIFO, in modo da stampare l'ultimo elemento inserito per primo).

Altrimenti, se il numero di caratteri nel prefisso i -esimo di T è maggiore o uguale al numero di caratteri nel prefisso j -esimo di P , allora la computazione si biforca ricorsivamente: o semplicemente ignoriamo l'ultimo carattere di T (considerando il prefisso $i-1$ -esimo), oppure, se i caratteri sono uguali, inseriamo l'indice i nello stack e chiamiamo la funzione ricorsivamente con $i-1$ e $j-1$.

```
printAll(ITEM[] T, int n, ITEM[] P, int m)
{
    STACK S = Stack()
    printAllRec(T, P, n, m, S)
}
```

```

printAll(ITEM[] T, ITEM[] P, int i, int j, STACK S)
  if j == 0 then
    | print S                                % Stampa in ordine LIFO
  else if i ≥ j then
    { Caso in cui si ignora l'ultimo carattere di P }
    printAllRec(T, P, i - 1, j, S)
    { Caso in cui si considera l'ultimo carattere di P, se sono uguali }
    if T[i] == P[j] then
      | S.push(i)
      | printAllRec(T, P, i - 1, j - 1, S)
      | S.pop

```

Nel caso le due stringhe siano composte da n ed m caratteri tutti uguali ad un carattere c , ogni chiamata ricorsiva può generare due chiamate ricorsive, con un costo limitato superiormente da $O(2^n)$. È importante notare tuttavia che tramite il meccanismo di pruning e tenuto conto che in input "normali" i caratteri saranno diversi, il numero di chiamate può essere molto inferiore, specialmente per valori di m vicini a 1 ed n .

Esercizio B3

La soluzione è basata su programmazione dinamica. Si costruiscono quattro matrici di appoggio, chiamate L , R , U , D (left, right, up, down). Le caselle $L[i][j]$, $R[i][j]$, $U[i][j]$, $D[i][j]$ contiene la più lunga fila di valori 1 che si estende a partire dalla cella (i, j) (compresa) nella direzione sinistra, destra, alto, basso.

Questi valori possono essere calcolati nel modo seguente:

$$\begin{aligned}
L[i][j] &= \begin{cases} M[i][j] & j = 1 \\ L[i][j-1] + 1 & i > 1 \wedge M[i][j] = 1 \\ 0 & i > 1 \wedge M[i][j] = 0 \end{cases} \\
R[i][j] &= \begin{cases} M[i][j] & j = n \\ R[i][j+1] + 1 & i < n \wedge M[i][j] = 1 \\ 0 & i < n \wedge M[i][j] = 0 \end{cases} \\
D[i][j] &= \begin{cases} M[i][j] & i = n \\ D[i+1][j] + 1 & j < n \wedge M[i][j] = 1 \\ 0 & j < n \wedge M[i][j] = 0 \end{cases} \\
U[i][j] &= \begin{cases} M[i][j] & i = 1 \\ U[i-1][j] + 1 & j > 1 \wedge M[i][j] = 1 \\ 0 & j > 1 \wedge M[i][j] = 0 \end{cases}
\end{aligned}$$

Una volta calcolati questi valori, la più grande croce centrata in i, j è data da $\min(L[i][j], R[i][j], U[i][j], D[i][j]) \cdot 4 - 3$ (la somma dei quattro bracci, a cui viene sottratto -3 perché l'elemento centrale è contato quattro volte invece di 1).

```

int cross(int[][] M, int n)
    int[][] L = new int[1...n][1...n]
    int[][] R = new int[1...n][1...n]
    int[][] U = new int[1...n][1...n]
    int[][] D = new int[1...n][1...n]
    for j = 1 to n do
        U[1][j] = M[1][j]
        D[n][j] = M[n][j]
    for i = 1 to n do
        L[i][1] = M[i][1]
        R[i][n] = M[i][n]
    for i = 2 to n do
        for j = 2 to n do
            L[i][j] = iff(M[i][j] == 0, 0, L[i][j - 1] + 1)
            U[i][j] = iff(M[i][j] == 0, 0, U[i - 1][j] + 1)
    for i = n - 1 downto 1 do
        for j = n - 1 downto 1 do
            R[i][j] = iff(M[i][j] == 0, 0, R[i][j + 1] + 1)
            D[i][j] = iff(M[i][j] == 0, 0, D[i + 1][j] + 1)
    int maxsofar = 0
    for i = 1 to n do
        for j = 1 to n do
            int size = min(L[i][j], R[i][j], U[i][j], D[i][j]) · 4 - 3
            maxsofar = max(maxsofar, size)
    return maxsofar

```

Questa procedura ha costo $\Theta(n^2)$. Tuttavia, esiste anche una soluzione molto semplice che, partendo da ogni posizione (i, j) , cerca la croce più grande, senza utilizzare strutture di appoggio.

```

int cross(int[][] M, int n)
    int maxsofar = 0
    for i = 1 to n do
        for j = 1 to n do
            if M[i][j] == 1 then
                int dim = largestCross(M, i, j, n)
                maxsofar = max(maxsofar, dim)
    return maxsofar

```

La procedura largestCross() parte dalla posizione i, j e cerca di espandere la croce in tutte le direzioni, controllando di non uscire dalla matrice e che tutti i quattro bracci abbiano un bit attivo a distanza k .

```

int largestCross(int[][] M, int i, int j, int n)
    int k = 1 % Lunghezza del braccio
    while i + k ≤ n and j + k ≤ n and i - k ≥ 1 and j - k ≥ 1 and
        M[i][j - k] + M[i][j + k] + M[i - k][j] + M[i + k][j] == 4 do
        k = k + 1
    return k - 1

```

Questa versione ha costo $\Theta(n^3)$.